An Object-Oriented Car Dealership

Student Workshop 4w

Version 7.1 Y

Project Description

In this project, you will build a mini, console-based dealership application. This app would sit on the desk of a sales manager at a car dealership.

Currently, users of the application will be able to:

1 - Find vehicles within a price range

2 - Find vehicles by make / model

3 - Find vehicles by year range

4 - Find vehicles by color

5 - Find vehicles by mileage range

6 - Find vehicles by type (car, truck, SUV, van)

7 - List ALL vehicles

8 - Add a vehicle

9 - Remove a vehicle

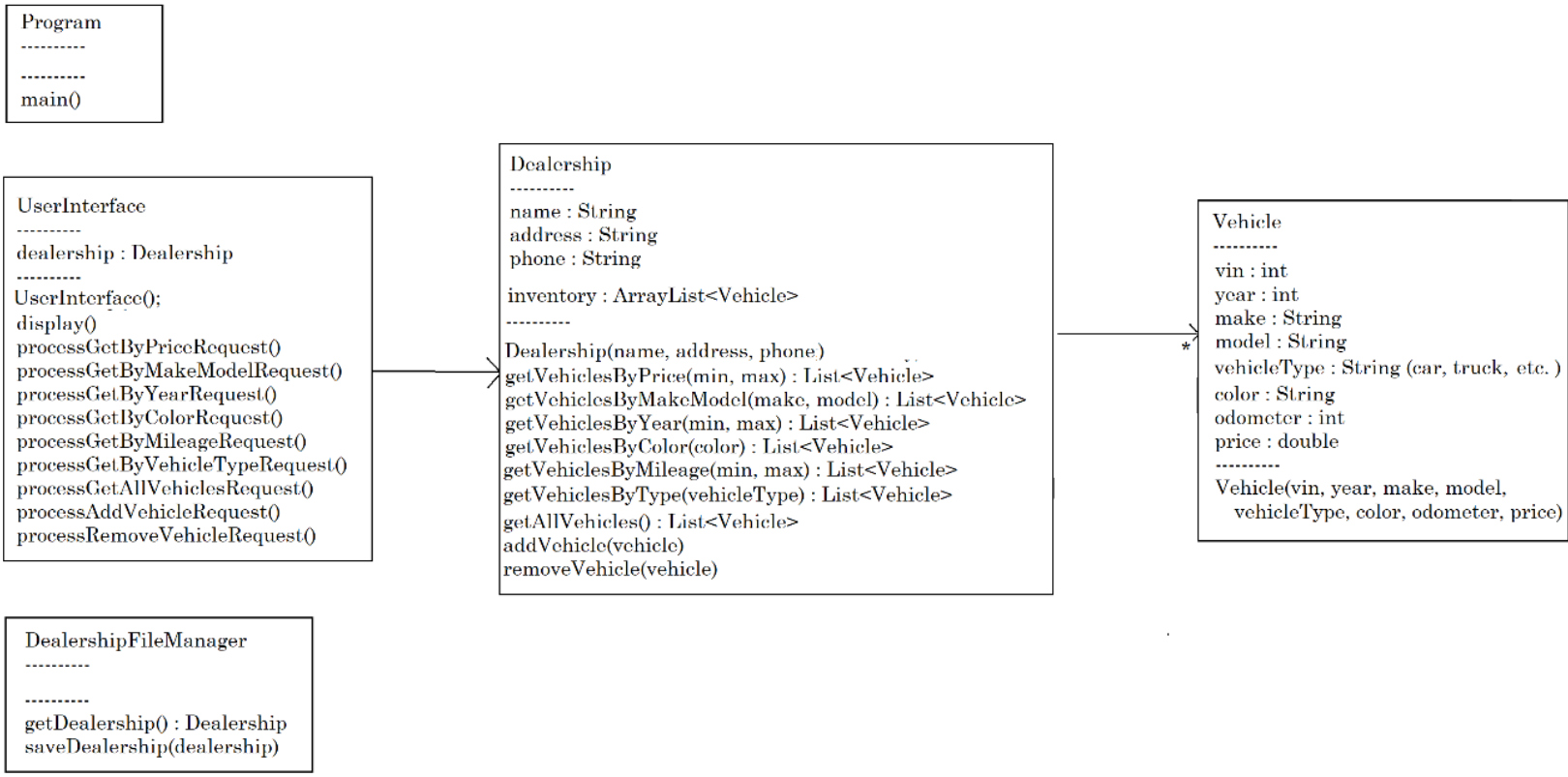
99 - Quit

The vehicles the dealership sells will need to be stored in a csv file delimited by |-pipes. Changes to the dealership's inventory will require the file to be updated. These changes can include adding and removing vehicles.

Project Details

You have worked on an application with this "theme" before. You are going to take that knowledge and create a new application with a more robust architecture.

Create the classes based on the diagram on the following page.


Program

---
main()
UserInterface
---
dealership : Dealership

UserInterface();
display()
processGetByPriceRequest()
processGetByMakeModelRequest()
processGetByYearRequest()
processGetByColorRequest()
processGetByMileageRequest()
processGetByVehicleTypeRequest()
processGetAllVehiclesRequest()
processAddVehicleRequest()
processRemoveVehicleRequest()
Dealership
---
name : String
address : String
phone : String
inventory : ArrayList<Vehicle>

Dealership(name, address, phone)
getVehiclesByPrice(min, max) : List<Vehicle>
getVehiclesByMakeModel(make, model) : List<Vehicle>
getVehiclesByYear(min, max) : List<Vehicle>
getVehiclesByColor(color) : List<Vehicle>
getVehiclesByMileage(min, max) : List<Vehicle>
getVehiclesByType(vehicleType) : List<Vehicle>
getAllVehicles() : List<Vehicle>
addVehicle(vehicle)
removeVehicle(vehicle)
*
Vehicle
---
vin : int
year : int
make : String
model : String
vehicleType : String (car, truck, etc.)
color : String
odometer : int
price : double

Vehicle(vin, year, make, model,
vehicleType, color, odometer, price)
DealershipFileManager

---
getDealership() : Dealership
saveDealership(dealership)

You will partition the logic into various classes.

Each class will be responsible for a different aspect of your app

- **Vehicle** will hold information about a specific vehicle
- **Dealership** will hold information about the dealership and maintain a list of vehicles. It will also have the methods to search, add, and remove vehicles from the list.
- **DealershipFileManager** will be responsible for reading the dealership file. It will parse the data, and create a Dealership object, and create each Vehicle to be added to the Dealership's inventory. It will also be responsible for saving the dealership and it's vehicles in the file in the same pipe-delimited format
- **UserInterface** will be responsible for all output to the screen, reading of user input, and "using the Dealership's search, add, and remove as needed. (ex: when the user selects "List all Vehicles", UserInterface would call the Dealership method and then display the vehicles it returns.)
- **Program** will be responsible for starting the application via its main() method and then creating the user interface and getting it started.

The inventory file for this application will be structured as follows:

- **The first line**: This will be the Dealerships information
- **The second line and on:** This will be each Vehicle in that dealerships inventory

Example:

```
D & B Used Cars|111 Old Benbrook Rd|817-555-5555
10112|1993|Ford|Explorer|SUV|Red|525123|995.00
37846|2001|Ford|Ranger|truck|Yellow|172544|1995.00
44901|2012|Honda|Civic|SUV|Gray|103221|6995.00
```

- This inventory structure is to enable a future feature, where we allow users to have access to more than one Dealership.
- This means, our main Program class, won't instantiate Dealerships. Our UserInterface will.

Programming Process

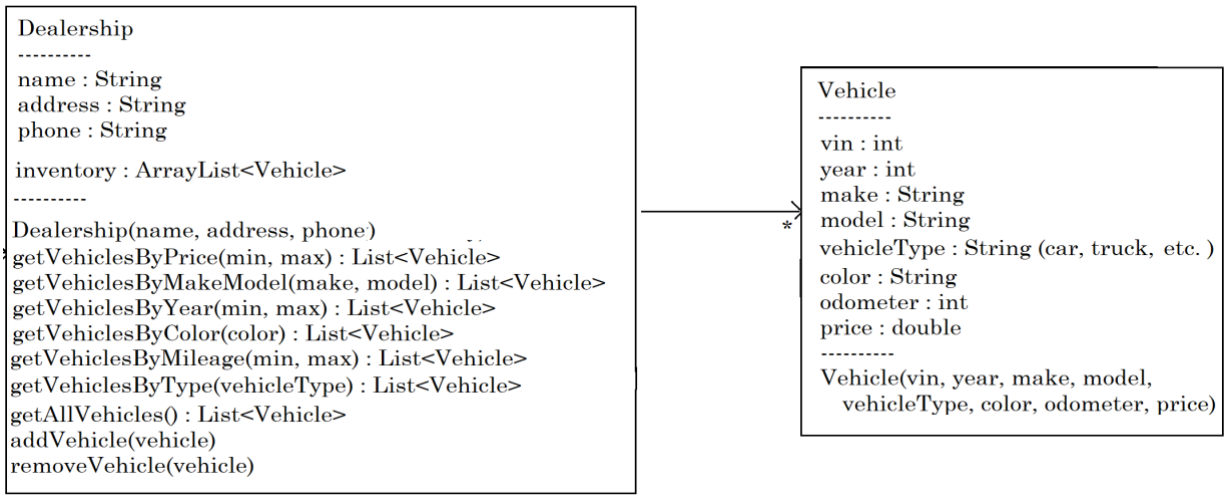
Begin by creating a new repo for this project on GitHub. Then clone it to your local machine in the `C:/pluralsight/workshops` directory.

Phase 1

First, start by coding the objects that will represent the data of your application.

The following UML diagram will help you figure out what you need:

It doesn't show any getters/setters, but you will need them for every field in the Vehicle class and every field in the Dealership class except for inventory.



Create these classes, but don't add any functionality to the methods except the constructor, the addVehicle() method, and getAllVehicles().

Note: Be sure to instantiate the ArrayList<Vehicle> object in the constructor.

All the search methods should return null. The remove() method should be empty. You can code and test the empty methods one at a time once you get all the infrastructure in place.

Compile and make sure there are no syntax errors. You won't be able to see any output just yet!

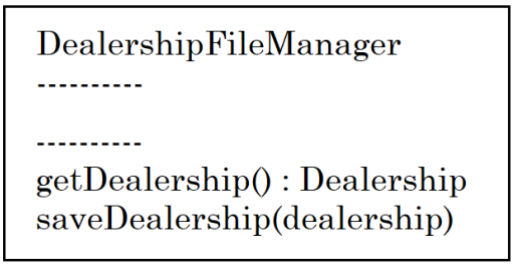
Commit, and push your code!

Phase 2

The next piece of code you will build is the persistence layer. This is the class that will understand how to read the dealership inventory file and convert the text into a fully loaded Dealership object.

It will also know how to take the data from a Dealership object and re-write/save its inventory file

The following UML diagram should help:



Before you code the class, create a starter data file. Then create a backup of it.

Now, code this class. Reflect on the data format you created earlier, this should help when creating this class!

Add code to `getDealership()` method. This method should load and read the `inventory.csv` file. Use the data in the file to create the `Dealership` object and populate the inventory with a list of `Vehicles`.

Create the `saveDealership()` method, but don't add any code to it. Focus on reading the dealership file and creating the Dealership object. This method will overwrite the inventory.csv file with the most current dealership information and inventory list.

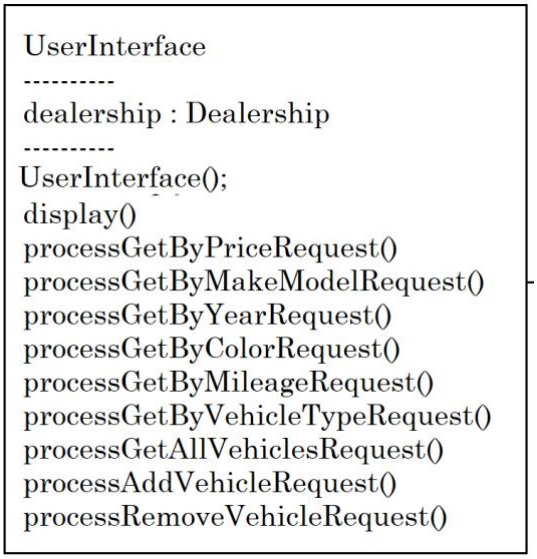
Compile and make sure there are no syntax errors. You won't be able to see any output just yet!

Commit and push your code!

Phase 3

The next piece of code you will build is the user interface layer. This is the class that will interact with the user and dispatch the commands to the back end of our application.

The following UML diagram should help you get started:



Code this class, and leave all the methods empty except for the following:

- **The display() method**
    - At the top of display(), call a private init() method. This not shown in the diagram above. It will load the dealership.
    - Then create a loop and within it to:
        - Display the menu
        - Read the user's command
        - Create a switch statement that calls the correct process() method that matches the user request

- **A private init() method**
    - In this method, you need to create your dealership object. In order to complete this:
        - Create an instance of your DealershipFileManager class
        - Call its getDealership() method
        - Assign the dealership that it returns to the UserInterface's this.dealership attribute
- **A private displayVehicles() helper method**
    - Because you will be displaying many different lists of vehicles, it makes sense to have a helper method that displays the list and can be called from all the search and get vehicles type of methods.
    - This method should have a parameter that is passed in containing the vehicles to list. Within the method, create a loop and display the vehicles!
- **The processAllVehiclesRequest() method**
    - In this method, we will list all vehicles in the dealership. The reason we code this in the first round is to see if we were able to successfully load the dealership and vehicles from the file. To code this method, you must:
        - Call the dealership's getAllVehicles() method
        - Call the displayVehicles() helper method passing it the list returned from getAllVehicles().

Compile and make sure there are no syntax errors. You won't be able to see any output just yet!

Commit and push your code!

Phase 4

Finally, in the Program class main() method, create an instance of the UserInterface class and call its display() method.

Compile and make sure there are no syntax errors.

Commit and push your code!

Phase 5

Now it's time to get into the "meat-and-potatoes" phase of coding… making everything work! There are two ways of going about it:

- Get all the list options working and then work on add and remove vehicles
- Get add and remove vehicles working and then work on all the list options

It's a personal preference. We would recommend doing the list options first because it could feel more successful getting a lot done.

But if we were worried about writing the functionality for add/remove, we might recommend those first!

IMPORTANT NOTE: Don't forget to have your UserInterface use the DealershipFileManager to save the dealership each time the user adds or removes a vehicle!

Compile, commit, and push your code!

What Makes a Good Workshop Project?

- **You should:**
  - Have a clean and intuitive user interface (give the user clear instructions on each screen)
  - Implement the ability for a customer to add/remove items to a cart and to purchase the items in the cart

- **You should adhere to best practices such as:**
  - Create a Java Project that follows the Maven folder structure
  - Create appropriate Java packages and classes
  - Class names should be meaningful and follow proper naming conventions (PascalCase)
  - Use good variable naming conventions (camelCasing, meaningful variable names)
  - Your code should be properly formatted easy to understand
  - use Java comments effectively

- **Make sure that:**
  - Your code is free of errors and that it compiles and runs

- **Build a PUBLIC GitHub Repo for your code**
  - Include a README.md file that describes your project